Project 3:-Maven Selenium Project – Automating Login Using Selenium

Objective

The goal of this project is to use Selenium WebDriver with Maven to automate the login process on the SauceDemo website using valid credentials and verify successful authentication.

Sample Website Used for Automation

The following website is used to carry out the automation:

https://www.saucedemo.com/

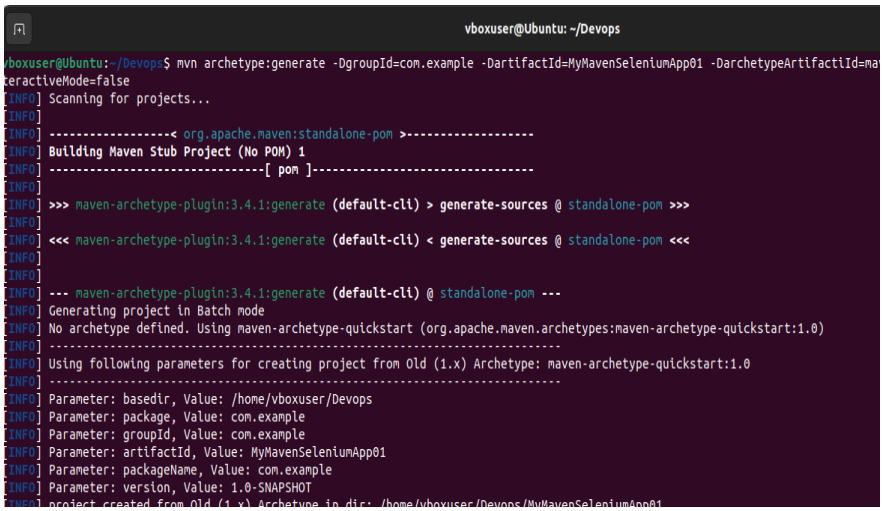
It provides a login page where credentials can be automated and validated through Selenium scripts.

Step 1: Create a Maven Project Using Terminal

Launch the terminal and use the command below to initialize a new Maven project:

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

This sets up the MyMavenSeleniumApp01 project with the standard Maven directory structure.



Project Structure:

```
vboxuser@Ubuntu:~/Desktop/Devops/Lab2/MyMavenSeleniumApp01$ tree
.
├── pom.xml
├── README.md
├── src
│   ├── main
│   │   └── java
│   │       └── com
│   │           └── example
│   │               └── App.java
│   └── test
│       └── java
│           └── com
│               └── example
│                   └── AppTest.java
└── target
    ├── classes
    │   └── com
    │       └── example
    │           └── App.class
    ├── generated-sources
    │   └── annotations
    ├── generated-test-sources
    │   └── test-annotations
    ├── maven-archiver
    │   └── pom.properties
    ├── maven-status
    │   └── maven-compiler-plugin
    │       ├── compile
    │       │   └── default-compile
    │       │       ├── createdFiles.lst
    │       │       └── inputFiles.lst
    │       └── testCompile
    │           └── default-testCompile
    │               ├── createdFiles.lst
    │               └── inputFiles.lst
    ├── MyMavenSeleniumApp01-1.0-SNAPSHOT.jar
    ├── surefire-reports
    │   ├── com.example.AppTest.txt
    │   └── TEST-com.example.AppTest.xml
    └── test-classes
        └── com
            └── example
                └── AppTest.class

28 directories, 14 files
vboxuser@Ubuntu:~/Desktop/Devops/Lab2/MyMavenSeleniumApp01$
```

Step 2: Install ChromeDriver

Selenium requires ChromeDriver to control the Chrome browser. Follow the steps below to install it.

Check Installed Chrome Version

Execute the command below in the terminal:

google-chrome --version

Example Output:

Google Chrome 133.0.6943.141

Extract Chrome Version

CHROME_VERSION=$(google-chrome --version | awk '{print $3}' | cut -d'.' -f1-3)

This reads the installed Chrome version and saves it in the CHROME_VERSION variable for use in subsequent steps.

Install jq (Used for JSON Parsing)

sudo apt install jq

Install curl (If Not Installed)

sudo apt install curl

Get Latest Compatible ChromeDriver Version

LATEST_DRIVER=$(curl -s "https://googlechromelabs.github.io/chrome-for-testing/latest-patch-versions-per-build.json" | jq -r ".builds[\"$CHROME_VERSION\"].version")

Download ChromeDriver

wget https://storage.googleapis.com/chrome-for-testing-public/$LATEST_DRIVER/linux64/chromedriver-linux64.zip

Extract ChromeDriver

unzip chromedriver-linux64.zip

Move ChromeDriver to System Path

sudo mv chromedriver-linux64/chromedriver /usr/local/bin/chromedriver

sudo chmod +x /usr/local/bin/chromedriver

Verify ChromeDriver Installation

chromedriver --version

Example Output:

ChromeDriver 133.0.6943.141

Confirm the displayed version matches the Chrome version installed on your system.

Step 3: Add Selenium Dependency in pom.xml

Edit pom.xml and insert the following Selenium dependency inside the <dependencies> tag:

```
<dependency>
<groupId>org.seleniumhq.selenium</groupId>
<artifactId>selenium-java</artifactId>
<version>4.29.0</version>
</dependency>
```

Step 4: Configure Maven Build Plugins

Insert the following plugin configuration inside the <build> section of pom.xml:

```
<build>
<plugins>

<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>

<configuration>
<source>1.8</source>
<target>1.8</target>
</configuration>

</plugin>

<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-jar-plugin</artifactId>
<version>3.0.1</version>

<configuration>
<archive>
<manifestEntries>
<Main-Class>com.example.App</Main-Class>
</manifestEntries>
</archive>
</configuration>
</plugin>
</plugins>
</build>
```

Open pom.xml ~/DevOps/MyMavenSeleniumApp01 Save

pom.xml App.java

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>MyMavenSeleniumApp01</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.29.0</version>
    </dependency>
  </dependencies>

</project>
```

Step 5: Inspect Elements in the Website

Before coding, identify the HTML elements on the page that Selenium will interact with.

Steps:

1. Visit https://www.saucedemo.com/ in the browser.
2. Right-click anywhere on the page to bring up the context menu.
3. Select Inspect from the menu to open the browser DevTools panel.
4. Use the element picker to hover over and highlight buttons on the page.

The key attributes used to identify the login form elements are:

user-name

password

login-button

The script uses these IDs to locate and interact with the login form elements on the page.

Step 6: Create App.java

Location:

src/main/java/com/example/App.java

Java Code:

```
package com.example;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class App {
  public static void main(String[] args) throws InterruptedException {
```

```
        WebDriver driver = new ChromeDriver();
        Thread.sleep(3000);

        // Open SauceDemo Login Page
        driver.get("https://www.saucedemo.com/");
        Thread.sleep(3000);

        // Maximize Browser Window
        driver.manage().window().maximize();
        Thread.sleep(3000);

        // Enter Username
        driver.findElement(By.id("user-name")).sendKeys("standard_user");
        Thread.sleep(3000);

        // Enter Password
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
        Thread.sleep(3000);

        // Click Login Button
        driver.findElement(By.id("login-button")).click();
        Thread.sleep(3000);

        // Close Browser
        driver.quit();
    }
}
```

```
vboxuser@Ubuntu: ~/Desktop/Devops/Lab2/MyMavenSeleniumApp01

[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< com.example:MyMavenSeleniumApp01 >------------------
[INFO] Building MyMavenSeleniumApp01 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyMavenSeleniumApp01 ---
[INFO] Deleting /home/vboxuser/Desktop/Devops/Lab2/MyMavenSeleniumApp01/target
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenSeleniumApp01 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vboxuser/Desktop/Devops/Lab2/MyMavenSeleniumApp01/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenSeleniumApp01 ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/vboxuser/Desktop/Devops/Lab2/MyMavenSeleniumApp01/target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenSeleniumApp01 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vboxuser/Desktop/Devops/Lab2/MyMavenSeleniumApp01/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenSeleniumApp01 ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/vboxuser/Desktop/Devops/Lab2/MyMavenSeleniumApp01/target/test-classes
[INFO]
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenSeleniumApp01 ---
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.009 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- maven-jar-plugin:3.0.1:jar (default-jar) @ MyMavenSeleniumApp01 ---
[INFO] Building jar: /home/vboxuser/Desktop/Devops/Lab2/MyMavenSeleniumApp01/target/MyMavenSeleniumApp01-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenSeleniumApp01 ---
[INFO] Installing /home/vboxuser/Desktop/Devops/Lab2/MyMavenSeleniumApp01/target/MyMavenSeleniumApp01-1.0-SNAPSHOT.jar to /home/vboxuser/.m2/repository/com/example/MyMavenSeleniumApp01/1.0-SNAPSHOT/MyMave
nSeleniumApp01-1.0-SNAPSHOT.jar
[INFO] Installing /home/vboxuser/Desktop/Devops/Lab2/MyMavenSeleniumApp01/pom.xml to /home/vboxuser/.m2/repository/com/example/MyMavenSeleniumApp01/1.0-SNAPSHOT/MyMavenSeleniumApp01-1.0-SNAPSHOT.pom
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.140 s
[INFO] Finished at: 2026-03-17T20:19:45+05:30
[INFO] ------------------------------------------------------------------------
vboxuser@Ubuntu:~/Desktop/Devops/Lab2/MyMavenSeleniumApp01$
```

Step 7: Build the Maven Project

Execute the command below in the terminal:

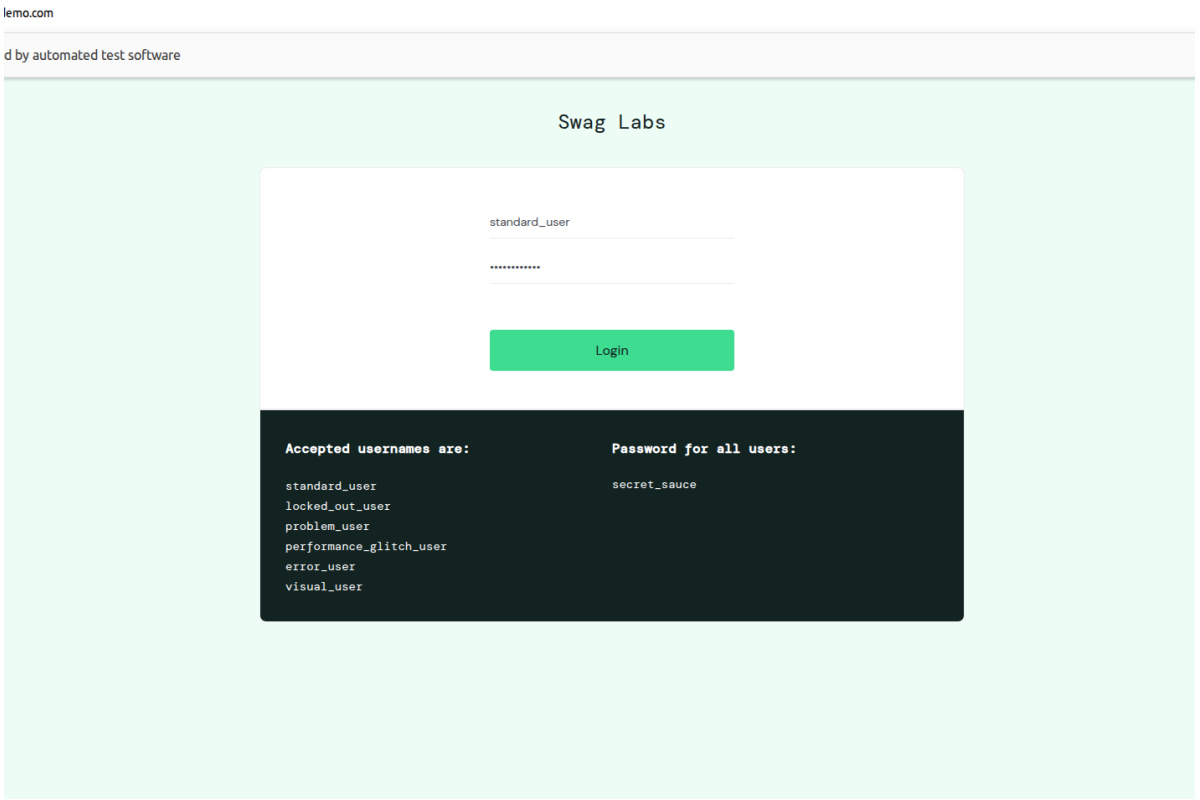
mvn clean install

Maven will compile the source files and package them into a JAR inside the target folder.

mvn compile

```
vboxuser@Ubuntu:~/Desktop/Devops/Lab2/MyMavenSeleniumApp01$ mvn exec:java -Dexec.mainClass="com.example.App"
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< com.example:MyMavenSeleniumApp01 >------------------
[INFO] Building MyMavenSeleniumApp01 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec-maven-plugin:3.6.3:java (default-cli) @ MyMavenSeleniumApp01 ---
[WARNING] thread Thread[UrlChecker-2,5,com.example.App] was interrupted but is still alive after waiting at least 14996msecs
[WARNING] thread Thread[UrlChecker-2,5,com.example.App] will linger despite being asked to die via interruption
[WARNING] thread Thread[CompletableFutureDelayScheduler,5,com.example.App] will linger despite being asked to die via interruption
[WARNING] NOTE: 2 thread(s) did not finish despite being asked to via interruption. This is not a problem with exec:java, it is a problem with the running code. Although not serious,
 it should be remedied.
[WARNING] Couldn't destroy threadgroup org.codehaus.mojo.exec.AbstractExecJavaBase$IsolatedThreadGroup[name=com.example.App,maxpri=10]
java.lang.IllegalThreadStateException
    at java.lang.ThreadGroup.destroy (ThreadGroup.java:776)
    at org.codehaus.mojo.exec.AbstractExecJavaBase.execute (AbstractExecJavaBase.java:345)
    at org.apache.maven.plugin.DefaultBuildPluginManager.executeMojo (DefaultBuildPluginManager.java:137)
    at org.apache.maven.lifecycle.internal.MojoExecutor.execute (MojoExecutor.java:210)
    at org.apache.maven.lifecycle.internal.MojoExecutor.execute (MojoExecutor.java:156)
    at org.apache.maven.lifecycle.internal.MojoExecutor.execute (MojoExecutor.java:148)
    at org.apache.maven.lifecycle.internal.LifecycleModuleBuilder.buildProject (LifecycleModuleBuilder.java:117)
    at org.apache.maven.lifecycle.internal.LifecycleModuleBuilder.buildProject (LifecycleModuleBuilder.java:81)
```

java -jar target/MyMavenSeleniumApp01-1.0-SNAPSHOT.jar/mvn exec:java -Dexec.mainClass=”com.example.App”


lemo.com
d by automated test software
Swag Labs
standard_user
Login
Accepted usernames are:
standard_user
locked_out_user
problem_user
performance_glitch_user
error_user
visual_user
Password for all users:
secret_sauce

Jenkins Pipeline for SeleniumApp1 (Auto-sausedemo)

Step 1: Open the Jenkins Dashboard in order to create a new item.

Step 2: Click on New Item in order to create a New Item. Give the name as "SeleniumApp1" and Item type as Pipeline, then click Create.

Step 3: Configure the settings by defining the Definition as "Pipeline Script from SCM" and SCM as "Git". Paste the Repository URL of the project from GitHub and choose Credentials as "GitHub Credentials" which was created earlier.

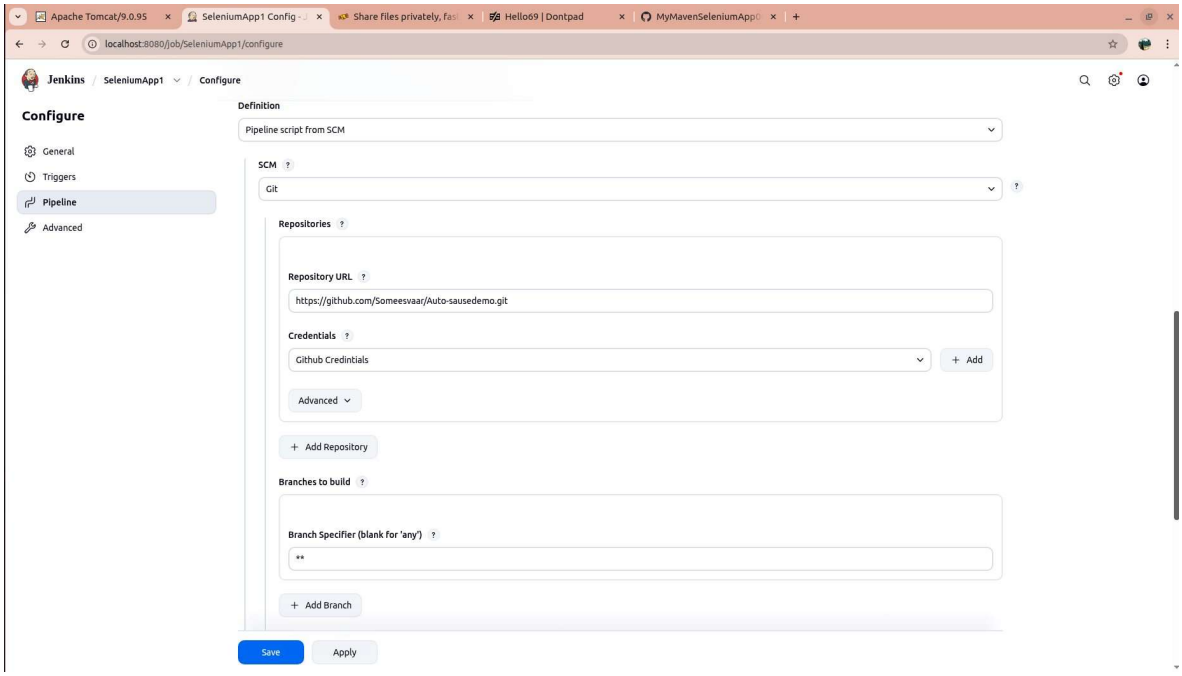


Fig A-1: Jenkins Pipeline configuration – SCM set to Git with Repository URL for Auto-sausedemo.git and GitHub Credentials

Step 4: In Terminal, go to the directory created for SeleniumApp1 and create a new file called "Jenkinsfile".

Step 5: Paste the following code inside the Jenkinsfile:

```
pipeline {
    agent any  // Use any available agent

    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
        jdk 'JDK'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/Someesvaar/Auto-sausedemo.git'
            }
        }

        stage('Build') {
```

```
            steps {
                sh 'mvn clean package'  // Run Maven build
            }
        }

        stage('Test') {
            steps {2
                sh 'mvn test'  // Run unit tests
            }
        }




        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenSeleniumApp01-1.0-SNAPSHOT.jar'
            }
        }



    }
    post {
        always {
            echo 'Pipeline completed.'
        }
    }
}
```

And add the following to App.java
options.addArguments("--headless=new"); // modern headless mode
options.addArguments("--no-sandbox"); // required in Jenkins/Linux
options.addArguments("--disable-dev-shm-usage"); // prevents crashes
options.addArguments("--disable-gpu");

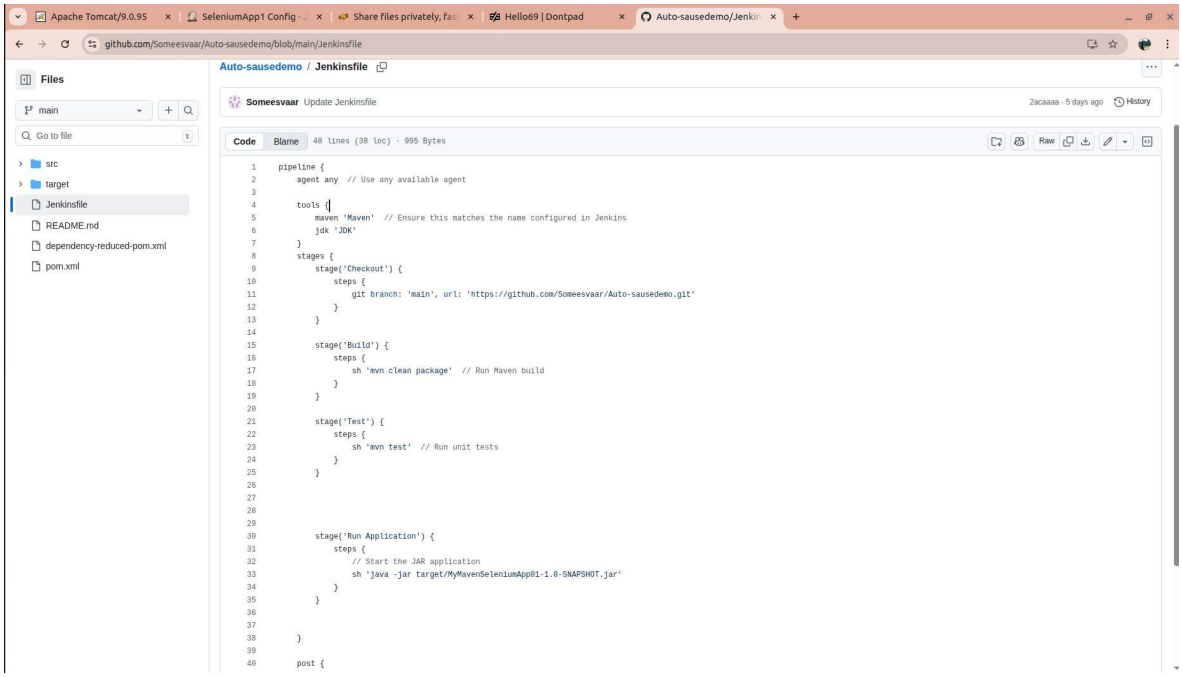


Fig A-2: Jenkinsfile committed to the Auto-sausedemo GitHub repository

Step 6: Update pom.xml to change the versions of source and target from 1.7 to 1.8.

Step 7: Push the changes and commit by pushing the "Jenkinsfile" to the respective repository on GitHub.

Step 8: Go to Jenkins and Build the Pipeline, then check the Stage View to confirm successful execution.

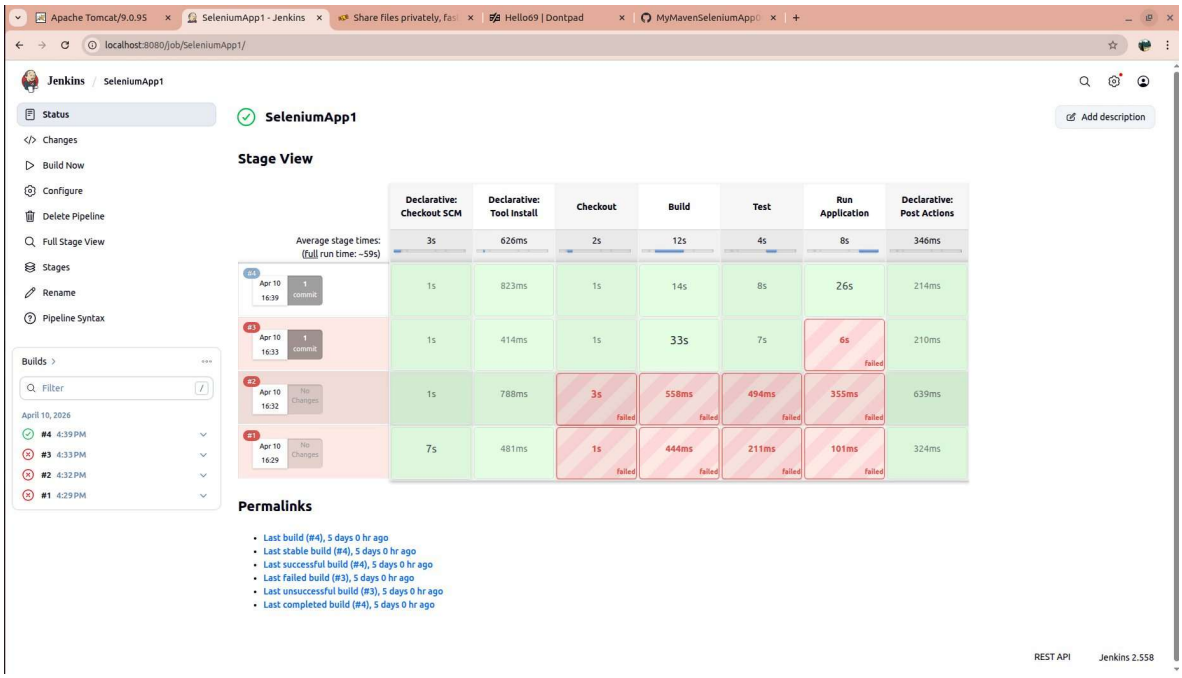


Fig A-3: Jenkins Stage View showing Build #4 successful for SeleniumApp1 (~59s total)